

Hierarchical Globally Coupled LDPC Codes and Adaptive Two-Phase Decoding for NAND Flash Memories

Mengxin Hu¹, Xiongfei Zhai¹, Shunyou Tao¹, and Guojun Han¹, *Senior Member, IEEE*

Abstract—The conventional low density parity check (LDPC) codes face the requirements of increasing reliability, lower decoding latency, reduced complexity in NAND flash memory. In this letter, a novel hierarchical globally-coupled LDPC (HC-GC-LDPC) code is proposed to address the above issue. By introducing a two-level coupled structure, the constructed (37632, 33712) HC-GC-LDPC code achieves a decoding performance gain of roughly 0.1 dB over the spatially-coupled LDPC (SC-LDPC) codes under additive white Gaussian noise (AWGN) channels. Furthermore, an adaptive fragmentation-aware two-phase decoding algorithm (AFA-TPA) is developed, which reduces the average check-node updates by about 3.5% compared to the conventional two-phase algorithm, while maintaining the decoding performance. Simulation results under the NAND flash channels demonstrate that the HC-GC-LDPC code maintains the superior performance in high raw-bit-error-rate (RBER) regions, illustrating its robustness and practical applicability for high-density flash memory systems.

Index Terms—LDPC codes, two-level coupled structure, adaptive decoding, NAND flash memory, two-phase algorithm.

I. INTRODUCTION

WITH the unprecedented growth of digital information, high-density storage technology has evolved rapidly. NAND flash memory, owing to its high storage density, energy efficiency, and read/write speed, has become the cornerstone of modern storage systems. Its architecture has advanced from 2D to 3D with multi-bit cells, but this density increase degrades reliability by elevating the raw bit error rate (RBER) and threatening data integrity [1]. To address these challenges, error characterization frameworks have been developed to model program/erase cycling effects [2] and systematic reliability issues [3]. Key contributions include inter-cell interference detection [4] and online flash channel modeling [5], with recent studies extending to 3D architectures. 3D stacking technology introduces further challenges such as inter-layer interference and process non-uniformity [6]. Recent investigations into 3D charge trap NAND characteristics and inter-cell interference [7] provide essential channel understanding for developing advanced error-correction solutions tailored to next-generation flash systems.

Received 20 April 2026; accepted 29 April 2026. Date of publication 5 May 2026; date of current version 13 May 2026. This work was supported in part by National Natural Science Foundation of China under Grants 62471151 and 62301166; in part by the Science and Technology Program of Guangzhou under Grant 2025A04J2904. The associate editor coordinating the review of this letter and approving it for publication was R. Asvadi. (Corresponding authors: Xiongfei Zhai; Guojun Han.)

The authors are with the School of Information Engineering, Guangdong University of Technology, Guangzhou 510006, China (e-mail: 850916216@qq.com; zhaixiongfei@gdut.edu.cn; 201977654@qq.com; gjhan@gdut.edu.cn).

Digital Object Identifier 10.1109/LCOMM.2026.3690323

To safeguard data integrity against rising error rates, powerful error correction codes (ECCs) are indispensable in NAND flash memories. Initially, Bose-Chaudhuri-Hocquenghem (BCH) codes with hard-decision decoding were sufficient for early flash memories [8], [9]. However, their limited correction capability falls short of the stringent reliability requirements for modern multi-level cell flash. Consequently, low-density parity-check (LDPC) codes have become the de facto ECC standard, owing to their powerful error-correction performance with soft-decision decoding. The industry trend is steadily moving towards longer codewords to achieve stronger reliability for future high-density storage applications [10], [11].

The design of structured LDPC codes tailored for the non-ideal NAND flash channel has been a focal point. Specifically, the family of globally-coupled LDPC (GC-LDPC) codes has emerged as a particularly promising architecture due to its inherent support for parallel, cooperative decoding. Recent research has further generalized GC-LDPC code constructions to enhance design flexibility and adaptability to diverse channel conditions [12]. A highly structured (21150, 19050) GC-LDPC code was designed specifically for flash memories in [13] a two-phase algorithm (TPA). To improve the connectivity of structure and reduce the complexity, by introducing a tail-biting structure, the tail-biting GC-LDPC (TB-GC-LDPC) codes were proposed in [14]. More recently, a spatially coupled GC-LDPC (SC-GC-LDPC) code was constructed in [15]. However, the exiting codes may not fully exploit hierarchical structural optimizations, potentially limiting their performance and applications.

To address the aforementioned challenges, this letter constructs a hierarchical GC-LDPC (HC-GC-LDPC) code, which extends the traditional GC-LDPC framework by incorporating a two-level coupled architecture. To harness the structural advantages of HC-GC-LDPC and adapt to the flash channel's characteristics, we propose an adaptive fragmentation-aware TPA (AFA-TPA), where the complexity is reduced by identifying the incorrect local codes and only activating the necessary subsets of global part.

The remainder of this letter is organized as follows. Section II details the construction of the proposed HC-GC-LDPC codes. Section III describes the AFA-TPA decoding algorithm. Section IV presents comprehensive experimental results and analysis.

II. CONSTRUCTION OF HC-GC-LDPC CODES

In this section, we first review the construction of GC-LDPC codes. Then, by introducing a multi-scale hierarchical coupled

strategy, we present the detailed construction of the proposed HC-GC-LDPC codes.

A. Construction of GC-LDPC Codes

The construction of GC-LDPC codes begins with the selection of a finite field $\text{GF}(q)$, where q is a prime power. Let α be a primitive element of $\text{GF}(q)$. The $q - 1$ non-zero elements of $\text{GF}(q)$ can then be represented as powers of α , i.e., $\{\alpha^0, \alpha^1, \dots, \alpha^{q-2}\}$, with $\alpha^{q-1} = \alpha^0 = 1$. Then the following $(q - 1) \times (q - 1)$ matrix $\mathbf{S}$ is formed based on $\text{GF}(q)$.

$$\mathbf{S} = \begin{bmatrix} \alpha^0 - 1 & \alpha - 1 & \cdots & \alpha^{q-2} - 1 \\ \alpha^{q-2} - 1 & \alpha^0 - 1 & \cdots & \alpha^{q-3} - 1 \\ \vdots & \vdots & \ddots & \vdots \\ \alpha - 1 & \alpha^2 - 1 & \cdots & \alpha^0 - 1 \end{bmatrix}. \quad (1)$$

We assume that $q - 1 = l \times r$, where l and r are positive integers. Then the parity-check matrix of GC-LDPC codes can be constructed by following these steps.

- 1) **Base Matrix Decomposition:** Firstly, the matrix $\mathbf{S}$ is divided into r sub-matrices of size $l \times (q - 1)$, denoted by $\mathbf{W}_0, \dots, \mathbf{W}_{r-1}$, which can construct the matrix $\mathbf{B}_1$ in (2). For the ease of presentation, we denote the set of sub-matrices by $\mathcal{W} = \{\mathbf{W}_0, \dots, \mathbf{W}_{r-1}\}$. Note that the size of $\mathbf{B}_1$ is larger than that of $\mathbf{S}$, facilitating the construction of subsequent matrices.

$$\mathbf{B}_1 = \begin{bmatrix} \mathbf{W}_0 & \mathbf{W}_1 & \cdots & \mathbf{W}_{r-1} \\ \mathbf{W}_{r-1} & \mathbf{W}_0 & \cdots & \mathbf{W}_{r-2} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{W}_1 & \mathbf{W}_2 & \cdots & \mathbf{W}_0 \end{bmatrix}. \quad (2)$$

- 2) **Local Part Construction:** By selecting the integers m and n with $m \leq l$ and $n \leq q - 1$, we can construct a smaller matrix $\mathbf{R}_k$ from $\mathbf{W}_k$, which consists of the first m rows and the first n columns of $\mathbf{W}_k$, forming the following matrix $\mathbf{B}_2$. Every $\mathbf{R}_k, k = 0, \dots, r - 1$, can be served as the local part of parity-check matrix. Note that $\mathbf{R}_k$ is full-rank with $m \leq l$ and $n \leq q - 1$.

$$\mathbf{B}_2 = \begin{bmatrix} \mathbf{R}_0 & \mathbf{R}_1 & \cdots & \mathbf{R}_{r-1} \\ \mathbf{R}_{r-1} & \mathbf{R}_0 & \cdots & \mathbf{R}_{r-2} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{R}_1 & \mathbf{R}_2 & \cdots & \mathbf{R}_0 \end{bmatrix}. \quad (3)$$

- 3) **Global Part Construction:** The remaining rows from all $\mathbf{W}_k$ constitute a pool of candidate rows for global coupling. From this pool, s rows ($1 \leq s \leq r \times (l - m)$) are selected. To form the global part $\mathbf{G}$, the first t contiguous segments (each of length n) are extracted from each of the s selected rows. This results in a global matrix $\mathbf{G}$ of size $s \times nt$. For simplicity, we select $\mathbf{R}_0$ as the local part. The overall parity-check matrix $\mathbf{H}$ is then assembled as shown in (4).

$$\mathbf{H} = \begin{bmatrix} \mathbf{R}_0 & & & \\ & \mathbf{R}_0 & & \\ & & \ddots & \\ & & & \mathbf{R}_0 \\ \hline & & & \mathbf{G} \end{bmatrix}. \quad (4)$$

For example, by setting $q = 197$, $\alpha = 3$, $l = 4$, $r = 49$, $m = 4$, $n = 48$, $s = 4$, and $t = 4$, a 4KB GC-LDPC code can be constructed with the local part being $\mathbf{R}_0$ and the global part being the fifth to eighth rows of $\mathbf{W}_0$.

To further optimize the code's performance, the technologies of masking and fold expansion can be exploited. For instance, an (33712, 37632) GC-LDPC code is developed accordingly with the code rate being 0.8958.

B. Construction of HC-GC-LDPC Codes

In this subsection, we propose the procedures of constructing the HC-GC-LDPC codes, where the global part is further divided into the global-local components and the global-global component.

The parity-check matrix $\mathbf{H}$ of the proposed HC-GC-LDPC code is constructed through the following procedures:

- 1) Construct $\mathbf{B}_1$ and $\mathbf{B}_2$ based on the procedures of constructing GC-LDPC codes.
- 2) **Parameter Definition:** The key structural parameters are defined, including the number of local blocks t , the size of global-local components ($\mathbf{G}_{L,k}, k = 1, \dots, u$) $f \times g$, the number of global-local components u , the number of rows in the global-global component ($\mathbf{G}_G$) v . Note that $g \times u = n \times t$ must be satisfied.
- 3) **Local Part Selection:** Select an arbitrary matrix $\mathbf{R}_k$ to serve as the local part..
- 4) **Global-Local Component Construction:** Construct $\mathbf{G}_{L,k}$ by selecting the rows and columns from the pool formed by the remaining rows after formulating $\mathbf{B}_2$, with the indexes of rows and columns being $f(k-1)+1$ to fk and 1 to g , respectively.
- 5) **Global-Global Component Construction:** Construct $\mathbf{G}_G$ similarly with the indexes of rows and columns being $fu+1$ to $fu+v$ and 1 to $t \times n$. Subsequently, the parity-check matrix of proposed HC-GC-LDPC code $\mathbf{H}$ is presented by

$$\mathbf{H} = \begin{bmatrix} \mathbf{R}_1 & & & \\ & \mathbf{R}_2 & & \\ & & \ddots & \\ & & & \mathbf{R}_t \\ \hline \mathbf{G}_1 & & & \\ & \mathbf{G}_2 & & \\ & & \ddots & \\ & & & \mathbf{G}_u \\ \hline \mathbf{G}_G \end{bmatrix}$$

As we can see, the proposed HC-GC-LDPC code enables a hierarchically coupled architecture, which degenerates to a traditional GC-LDPC code if $t = u$. Specifically, the HC-GC-LDPC code can be flexibly designed by considering different u and v to achieve the independent adjustment of coupling granularity and coupling strength.

C. Examples of 4KB HC-GC-LDPC Codes Over $\text{GF}(197)$

To explore the trade-offs between we present three examples of 4KB HC-GC-LDPC codes over $\text{GF}(197)$ in this subsection,

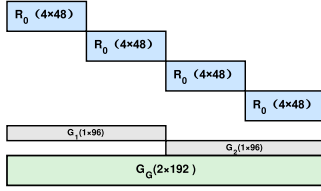


Fig. 1. The parity-check matrix of Example 1.

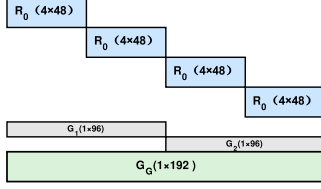


Fig. 2. The parity-check matrix of Example 2.

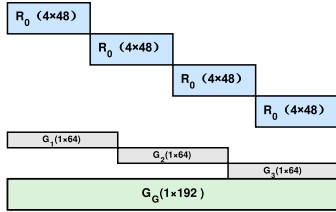


Fig. 3. The parity-check matrix of Example 3.

where the first one serves as the baseline, the second one is with lower column weight, and the third one aims at increasing the code rate while maintaining the low column weight. We consider $t = 4$ for all examples according to [15].

1) *Example 1:* With $l = 4$, $r = 49$, $m = 4$, $n = 48$, $t = 4$, $f = 1$, $g = 96$, $u = 2$, $v = 2$, we construct the global-local components $\mathbf{G}_{L,1}$ and $\mathbf{G}_{L,2}$ with the first two rows and the first 96 columns of the pool. Then $\mathbf{G}_G$ is formed by the third and fourth rows and the first 192 columns. Without loss of generality, $\mathbf{R}_0$ is selected as the local part. By exploiting the masking operation and a lifting factor of 196, the parity-check matrix of size 37632×33714 is created with the code rate being 0.895, as shown in Fig. 1.

As we can see, the column weight of 6 in the above parity-check matrix provides substantial connectivity. However, it also leads to the following practical limitations: (i) The expressive capacity of code can be affected, where the diversity of error patterns that can be corrected may be limited; (ii) The instability during iterative decoding may become worse; (iii) The susceptibility of overfitting may be increased.

To address these issues, we optimize the column weight in the second example.

2) *Example 2:* In this example, we reduce v to be 1, achieving a lower column weight of 5 and a higher code rate of 0.901. The optimized parity-check matrix is shown in Fig. 3 with the size of 37632×33908 .

Although Example 2 addresses the issue of high column weight, in the critical channel condition, we still need a lower code rate to ensure the error correction performance. Hence, in the following, the third example of HC-GC-LDPC code is presented with the code rate being 0.895 and the column weight being 5.

3) *Example 3:* In this example, we increase u from 2 to 3, while fixing $v = 1$. The resulting parity-check matrix of size 37632×33712 is shown in Fig. 2.

These three HC-GC-LDPC examples illustrate a design space exploration tailored for storage applications. Example 1 adopts $u = 2$, $v = 2$, resulting in a relatively high column weight that provides strong connectivity and benefits initial convergence, but may lead to a higher error floor—a critical concern in flash memory where residual errors must be extremely low. Example 2 reduces the column weight by adjusting v , aiming to lower the error floor and improve decoding stability, which is crucial for reliable data retrieval after extensive program/erase (P/E) cycles. Example 3 balances performance by increasing the number of global-local components u , thereby distributing global connections more evenly while maintaining a manageable column weight. This structure is expected to achieve a steep waterfall region for efficiency in the moderate RBER region and a low error floor for ultimate reliability in the late stage of the NAND flash memory service life, making it particularly suitable for the asymmetric and non-Gaussian NAND flash channel. The performance of the considered examples under both AWGN and flash channel conditions will be validated in Section IV.

III. ADAPTIVE FRAGMENTATION AWARE TWO-PHASE ALGORITHM

In this section, we propose AFA-TPA to improve the efficiency of decoding, which can adaptively select the necessary subset of global check nodes in the global phase. By applying this strategy, the average number of check node updates (CNU) can be significantly reduced.

The AFA-TPA requires the following inputs: The channel log-likelihood ratio (LLR) values, the parity-check matrix $\mathbf{H}$, the maximum iteration numbers I_L and I_G for the local and global phases, and the overall maximum iteration number I_{max} .

During the local phase, a 4×1 vector $\mathbf{f}$ is considered to denote the decoding results of subcodes, where $\mathbf{f}(t') = 1$ means the t' subcode does not pass the parity-check constraints. Within the iterations, only the failed subcodes undergo the parallel decoding utilizing their local parity-check constraints. Then $\mathbf{f}$ will be updated based on the decoding results. If all subcodes are decoded successively, i.e., $|\mathbf{f}|_1 = 0$, the AFA-TPA will be terminated immediately and output the decoded bits.

If there exist any subcodes failing to decode correctly within I_L iterations, the global phase is activated. Note that in this phase only the subset of global parity check matrix connecting the failed subcodes and their neighbors is exploited to perform decoding within I_G iterations. If the global decoding fails, the local phase will be re-activated with the obtained LLR values from the global phase. The total iteration number will not exceed I_{max} .

To improve the decoding efficiency and reduce the complexity, we propose the following adaptive strategy to select different subsets of global matrix based on $\mathbf{f}$.

If $|\mathbf{f}|_1 = 1$, i.e., the case with a single error subcode, we can only apply a minimal subset of global check matrix. For

Algorithm 1 AFA-TPA**Require:** Initial channel LLR values, $\mathbf{H}$, I_L , I_{max}

```

1: Initialize  $\mathbf{f} = (1, 1, 1, 1)$  and  $i = 1$ 
2: while  $i < I_{max}$  do
3:   Local Phase:
4:   for  $i_L = 1$  to  $I_L$  do
5:     Decode the failed subcodes in parallel
6:     if  $|\mathbf{f}|_1 == 0$  then
7:       break
8:     end if
9:   end for
10:  if  $|\mathbf{f}|_1 \neq 0$  then
11:    Adaptive Global Phase:
12:    Select different decoding patterns based on  $\mathbf{f}$  and
    perform the global decoding.
13:  end if
14:   $i = i + 1$ 
15: end while
16: return the decoded bits.

```

the aforementioned Example 1, if the first subcode fails, we can only select $\mathbf{G}_1$ and $\mathbf{G}_G$.

If $|\mathbf{f}|_1 = 2$, the selected subset depends on the positions of these two failed subcodes. For Example 1, if the first two subcodes fail, only $\mathbf{G}_1$ and $\mathbf{G}_G$ are selected. However, if the second and third subcodes fail, the complete global check matrix needs to be exploited.

If $|\mathbf{f}| \geq 3$, we need to select the full global check matrix, including $\mathbf{G}_1$, $\mathbf{G}_2$ and $\mathbf{G}_G$.

The activated subset preserves all essential global constraints for the failed subcodes, ensuring that the decoding subgraph retains sufficient connectivity and convergence is guaranteed by the same mechanism as conventional TPA. Moreover, the proposed adaptive selection strategy links decoding complexity to the error distribution. For the common cases in flash memory where only one or two subcodes fail, AFA-TPA significantly reduces the average number of check node updates (CNU). The complete AFA-TPA is summarized in Algorithm 1.

IV. EXPERIMENT RESULT

In this section, we evaluate the error correction performance of the proposed HC-GC-LDPC code under both AWGN and NAND flash memory channels via Monte Carlo simulations. The NAND flash channel model is established by adopting the widely recognized statistical characteristics from prior studies [15], which realistically capture the threshold voltage distribution, the program interference, and the retention noise.

As the benchmarks, we consider the 4KB GC-LDPC code described in Section II and the SC-GC-LDPC code in [15]. For decoding, we employ the normalized min-sum (NMS) algorithm with the normalization factor and the maximum iteration number being 0.7 and 20. We consider $(I_L, I_G, I_{max}) = (3, 2, 7)$, which is comparable with the NMS algorithm in terms of CNU. This complexity alignment strategy is consistent with the approach adopted in [15].

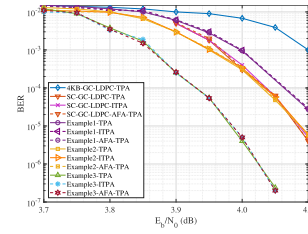


Fig. 4. BER performance versus E_b/N_0 under AWGN channels.

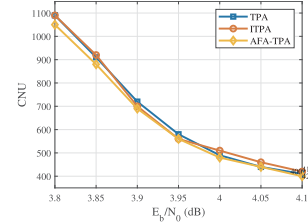


Fig. 5. Average number of CNUs versus E_b/N_0 in AWGN channels for Example 3.

A. Decoding Performance in AWGN Channels

Fig. 4 presents the bit error ratio (BER) performance of considered LDPC codes versus E_b/N_0 in AWGN channels. Specifically, different decoding algorithms (i.e., TPA, ITPA, and AFA-TPA) are exploited and the GC-LDPC code only employs TPA due to its structure. The proposed HC-GC-LDPC codes, especially Example 3, demonstrate a significantly steeper waterfall region and achieve the lower BER among all evaluated codes. One can see that when BER is 10^{-5} , Example 3 outperforms the SC-GC-LDPC code with a gap of roughly 0.1 dB, where the performance gain is more pronounced as compared with the 4KB GC-LDPC code. This improvement is mainly attributed to the hierarchical coupled structure, which enhances the connectivity between the local and global check nodes and facilitates more reliable message propagation during iterative decoding. Furthermore, the performance superiority of HC-GC-LDPC codes remains consistent within the considered SNR region revealing their robust error-correction capability under AWGN channels.

To compare the complexity of different algorithms, Fig. 5 shows their average numbers of CNUs versus E_b/N_0 . As we can see, the proposed AFA-TPA reduces the average number of CNU by roughly 2.72% and 3.49% compared to ITPA and TPA, respectively. Combining Figs. 4 and 5, we can conclude that the proposed AFA-TPA can efficiently reduce the decoding complexity without compromising the BER performance.

B. Performance in NAND Flash Channels

Fig. 6 presents the BER performance of considered LDPC codes versus RBER in NAND flash channels. One can see that, their BER performance degrades with the RBER increasing. Besides, Example 3 outperforms Examples 1 and 2, primarily due to their different coupling structure for the non-ideal flash channels. Example 1, due to its high column weight, exacerbates message correlation in the Tanner graph, resulting in inferior decoding performance. By reducing the column weight, Example 2 consequently weakens the global

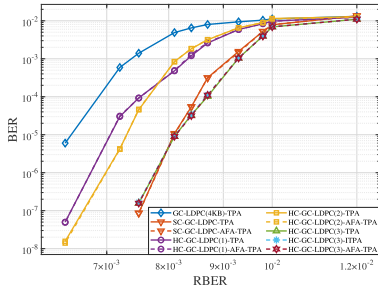


Fig. 6. BER performance versus RBER under NAND flash channels.

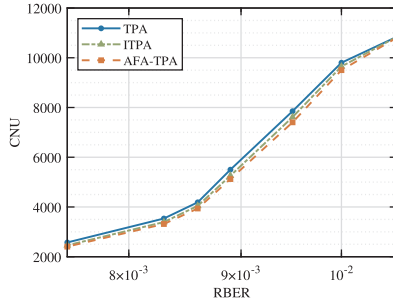


Fig. 7. Average number of CNUs versus RBER for Example 3. under NAND flash channels.

TABLE I
KEY DEGREE DISTRIBUTIONS AND EXIT ANALYSIS OF
CONSIDERED LDPC CODES

Code	Edges	Avg. VN Deg	Avg. CN Deg	Decoding threshold	No. of 6-cycle
GC-LDPC	262640	6.98	67.00	3.8934	5.31×10^7
Example 1	225204	5.98	57.45	3.8137	1.33×10^7
Example 2	187768	4.99	50.42	3.7975	3.57×10^6
Example 3	187768	4.99	47.90	3.7115	2.77×10^6
SC-GC-LDPC	211416	5.63	54.71	3.8024	6.46×10^6

cooperative error correction capability. In contrast, Example 3 features a chain-like message propagation mechanism, which more effectively suppresses the burst errors in flash channels, thereby exhibiting stronger robustness in high RBER regions. Specifically, within the practical RBER ranging from 8×10^{-3} to 1.2×10^{-2} , Example 3 achieves the lowest BER. As shown in Figs. 4 and 6, for the proposed HC-GC-LDPC structure, a larger u improves the performance, while a larger v brings degradation.

Fig. 7 illustrates the number of CNUs of different algorithms versus RBER. We can see that the decoding complexity increases significantly as the channel condition deteriorates. However, AFA-TPA remains the lowest complexity within the considered RBER region. Specifically, the average number of CNUs for AFA-TPA is reduced by approximately 1.8% compared to ITPA and by 4.1% compared to the conventional TPA. It is mainly due to the fact that the probability where only one or two subcodes fail is pretty high for common NAND flash memory.

C. Structural and Theoretical Analysis

Table I shows the key degree distributions and EXIT analysis of considered LDPC codes. As we can see, Example 3 achieves the smallest total number of edges, the lowest average variable node degree, and the lowest average check node degree, indicating the sparsest matrix, the lowest decoding complexity, and the highest accuracy of check information.

This coincides with the results in Figs. 4 and 6. Moreover, Example 3 achieves a lower decoding threshold and exhibits significantly fewer short cycles than GC-LDPC and SC-GC-LDPC, which further reduces the message correlation and improves the convergence.

V. CONCLUSION

This letter proposed a novel HC-GC-LDPC code and an AFA-TPA for high-density NAND flash memories. The constructed LDPC codes introduced a two-level coupling architecture, enabling the flexible configuration for the dynamic channels. Simulation results shown that our proposed codes outperform the existing codes in both AWGN and NAND flash channels to achieve lower BER, with a performance gap of roughly 0.1 dB. Besides, it was shown that the average number of CNU can be reduced by applying the proposed AFA-TPA as compared to the conventional algorithms, where the gap is up to 4.1%.

REFERENCES

- [1] D. Kang et al., "256 Gb 3 b/Cell V-NAND flash memory with 48 stacked WL layers," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 210–217, Jan. 2017.
- [2] Y. Cai, E. F. Haratsch, O. Mutlu, and K. Mai, "Threshold voltage distribution in MLC NAND flash memory: Characterization, analysis, and modeling," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2013, pp. 1285–1290.
- [3] Y. Cai, S. Ghose, E. F. Haratsch, Y. Luo, and O. Mutlu, "Error characterization, mitigation, and recovery in flash-memory-based solid-state drives," *Proc. IEEE*, vol. 105, no. 9, pp. 1666–1704, Sep. 2017.
- [4] M. Asadi, X. Huang, A. Kavcic, and N. P. Santhanam, "Optimal detector for multilevel NAND flash memory channels with intercell interference," *IEEE J. Sel. Areas Commun.*, vol. 32, no. 5, pp. 825–835, May 2014.
- [5] Y. Luo, S. Ghose, Y. Cai, E. F. Haratsch, and O. Mutlu, "Enabling accurate and practical online flash channel modeling for modern MLC NAND flash memory," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 9, pp. 2294–2311, Sep. 2016.
- [6] K.-T. Park et al., "Three-dimensional 128 gb MLC vertical NAND flash memory with 24-WL stacked layers and 50 MB/s high-speed programming," *IEEE J. Solid-State Circuits*, vol. 50, no. 1, pp. 204–213, Jan. 2015.
- [7] F. Wu et al., "Characterizing 3D charge trap NAND flash: Observations, analyses and applications," in *Proc. IEEE 36th Int. Conf. Comput. Design (ICCD)*, Oct. 2018, pp. 381–388.
- [8] R. C. Bose and D. K. Ray-Chaudhuri, "On a class of error correcting binary group codes," *Inf. Control*, vol. 3, no. 1, pp. 68–79, Mar. 1960.
- [9] J. Li, S. Lin, K. Abdel-Ghaffar, W. E. Ryan, and D. J. Costello, "Globally coupled LDPC codes," in *Proc. Inf. Theory Appl. Workshop (ITA)*, Jan. 2016, pp. 1–10.
- [10] L. Song, S. Yu, and Q. Huang, "Low-density parity-check codes: Highway to channel capacity," *China Commun.*, vol. 20, no. 2, pp. 235–256, Feb. 2023.
- [11] K. Zhao, W. Zhao, H. Sun, T. Zhang, X. Zhang, and N. Zheng, "LDPC-in-SSD: Making advanced error correction codes work effectively in solid state drives," in *Proc. USENIX Conf. File Storage Technol. (FAST)*, 2013, pp. 243–256.
- [12] Y.-C. Liao, H.-C. Chang, and S. Lin, "Generalized globally-coupled low-density parity-check codes," in *Proc. IEEE Inf. Theory Workshop (ITW)*, Nov. 2018, pp. 1–5.
- [13] B. Bao, Q. Li, W. Guan, Q. Wang, L. Liang, and X. Qiu, "Adaptive granularity progressive LDPC decoding for NAND flash memory," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 4, pp. 1312–1316, Apr. 2024.
- [14] J. Zhang, B. Bai, S. Li, M. Zhu, and H. Li, "Tail-biting globally-coupled LDPC codes," *IEEE Trans. Commun.*, vol. 67, no. 12, pp. 8206–8219, Dec. 2019.
- [15] J. Mo, X. Zhai, M. Yu, Y. Fang, and G. Han, "SC-GC-LDPC for NAND flash memory: Construction and decoding," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 11, pp. 4167–4180, Nov. 2025.